

sysfs, Uevents und udev

Wie Kernelzustand sichtbar wird, Geräteänderungen gemeldet und User-Space-Policy angewendet wird

4.03

Kernel- und Userspace haben getrennte Verantwortungen

01 Kernelzustand

Der Driver Core hält Geräte, Beziehungen und Attribute konsistent.

02 Ereignismeldung

Uevents transportieren Änderungen als strukturierte Schlüsselwerte.

03 Policy im Userspace

udev ordnet Namen, Rechte, Symlinks und weitere Eigenschaften zu.

04 Anwendungen

Programme lesen sysfs oder nutzen stabile Gerätepfade und APIs.

Kernel beschreibt Mechanismus · udev setzt distributions- und systemspezifische Policy um

Fünf Ebenen verbinden Zustand, Ereignis und Policy

Ebene	Form	Verwendung
Device Model	Kernelobjekte und Beziehungen	Quelle des aktuellen Zustands
sysfs	Dateien, Verzeichnisse, Symlinks	Zustand und steuerbare Attribute
Uevent	ACTION plus KEY=VALUE	flüchtige Änderungsnachricht
udev-Regel	Match und Zuweisungen	lokale Benennung und Rechte
/dev und Anwendungen	Geräteknoten und stabile Symlinks	Nutzung durch den User Space

Zustand ist lesbar; Änderungen sind Ereignisse; Policy bleibt außerhalb des Kernels.

Ein Gerätewechsel durchläuft Zustand, Ereignis und Policy



Zustand

sysfs bildet das registrierte Device Model ab

Nachricht

Uevent meldet Aktion und Geräteumgebung

Policy

udev wertet Regeln aus und aktualisiert User-Space-Sicht

Zeitbezug

Ein Uevent ist ein Zeitpunkt; sysfs zeigt den jeweils aktuellen Zustand.

sysfs projiziert dasselbe Device Model in mehrere Sichten

Pfad	Bedeutung	Typische Navigation
/sys/devices	Parent-Child-Hierarchie	Quelle der realen Device-Pfade
/sys/bus	Geräte und Treiber je Bustyp	devices/ und drivers/
driver-Symlink	aktuell gebundener Treiber	readlink -f ../driver
/sys/class	Sicht nach Gerätefunktion	Symlink zur Gerätehierarchie
/sys/dev	Lookup nach Major:Minor	char/ und block/

Symlinks sind Teil der Projektion: Sie verbinden funktionale, Bus- und Topologiesicht.

Ein sysfs-Attribut ist eine kleine textuelle ABI

```
static ssize_t status_show(struct device *dev,
                          struct device_attribute *attr,
                          char *buf)
{
    struct demo *d = dev_get_drvdata(dev);
    return sysfs_emit(buf, "%s\n", d->ready ? "ready" :
"idle");
}

static DEVICE_ATTR_RO(status);
```

- | | | |
|----|----------------|---|
| 01 | Name | stabiler Dateiname mit klarer Semantik |
| 02 | show() | formatiert aktuellen Zustand als Text |
| 03 | store() | validiert und übernimmt erlaubte Eingaben |
| 04 | Sync | Zugriff gegen remove und parallele Updates schützen |
| 05 | ABI | Änderungen können User Space brechen |

Attribute sollen klein, vorhersehbar und unabhängig von interaktiver Formatierung bleiben.

show, store und Gruppen strukturieren Attribute

show

- Zustand lesen
- sysfs_emit verwenden
- Newline anhängen
- Fehler zurückgeben



store

- Eingabe parsen
- Bereich validieren
- Zustand synchronisieren
- count zurückgeben



Attributgruppe

- zusammengehörige Dateien
- atomar veröffentlichen
- Sichtbarkeit steuern
- symmetrisch entfernen

Attribute sind Schnittstellen – keine Debug-Ausgabe und kein Ersatz für komplexe Protokolle.

Fünf Regeln halten sysfs-Attribute wartbar

eine Semantik

pro Datei ein klar abgegrenzter Wert oder Zustand

Textformat

maschinenlesbar, dokumentiert und ohne Dekoration

abschließender Newline

Ausgabe folgt üblichen Textdateikonventionen

Eingaben validieren

Parser, Wertebereich und Berechtigung prüfen

kurze Operation

keine lang laufende Arbeit im Attributzugriff

Ein sichtbares Attribut ist eine User-Space-Schnittstelle und braucht eine stabile, dokumentierte Semantik.

Attributgruppen sichern Publizieren und Rückbau

```
static struct attribute *demo_attrs[] = {
    &dev_attr_status.attr,
    &dev_attr_mode.attr,
    NULL,
};

static const struct attribute_group demo_group = {
    .attrs = demo_attrs,
};

ret = devm_device_add_group(dev, &demo_group);
```

- | | | |
|----|---------------------|---|
| 01 | Deklaration | DEVICE_ATTR_*
beschreibt Name, Rechte
und Callbacks |
| 02 | Gruppe | Attribute logisch
zusammenfassen |
| 03 | Publizieren | erst nach gültigem
Zustand freigeben |
| 04 | Rückbau | Managed API entfernt
die Gruppe beim
Detach |
| 05 | Sichtbarkeit | optionale Attribute
gezielt ein- oder
ausblenden |

Attribute dürfen nicht länger sichtbar bleiben als der Zustand, auf den ihre Callbacks zugreifen.

Ein Uevent transportiert Aktion und Geräteumgebung

Ereignisfelder

ACTION

DEVPATH

SUBSYSTEM

KEY=VALUE

Umgebung wird als
Schlüsselwerte übertragen

Bedeutung

Identität

DEVNAME · MAJOR · MINOR

Treiberhinweis

MODALIAS und Bus-Eigenschaften

Nach dem Ereignis ist sysfs die maßgebliche Sicht auf den aktuellen Zustand.

Der Kernel sendet standardisierte und ergänzte Uevents

```
char *envp[] = {  
    "STATE=ready",  
    NULL,  
};  
  
kobject_uevent_env(&dev->kobj, KOBJ_CHANGE, envp);  
  
/* Standardaktionen:  
 * add, remove, change, move,  
 * online, offline, bind, unbind  
 */
```

01 Aktion

KOBJ_ADD, KOBJ_REMOVE oder KOBJ_CHANGE wählen.

02 Umgebung

Nur notwendige, dokumentierte Schlüsselwerte ergänzen.

03 Zeitpunkt

Zustand vor dem Ereignis konsistent aktualisieren.

Uevents sind Benachrichtigungen – weder persistente Warteschlange noch Ersatz für lesbaren Zustand.

udev-Regeln trennen Matchbedingungen von Zuweisungen

```
ACTION=="add", SUBSYSTEM=="block", \
    KERNEL=="sd*", ENV{ID_BUS}=="usb", \
    SYMLINK+="lab/%k", MODE="0660"

# Match:
# ACTION, SUBSYSTEM, KERNEL, ENV{}, ATTR{}
# Zuweisung:
# NAME, SYMLINK, MODE, OWNER, GROUP, TAG
```

1 Event auswählen

Aktion, Subsystem und Kernelname eingrenzen

2 Merkmale prüfen

Uevent- oder sysfs-Werte vergleichen

3 Policy zuweisen

Symlink, Rechte, Tags oder Variablen setzen

4 kurz bleiben

lang laufende Dienste separat starten

Regeln werden auf Systemkontext angewendet: Rechte und Namensräume sind Teil des Sicherheitsmodells.

udev-Regeln lesen Ereignis- und Gerätekontext

Schlüssel	Quelle	Einsatz
ACTION	Uevent	add, remove, change ...
SUBSYSTEM	Uevent	block, net, tty, input ...
KERNEL	Gerätename	sd*, ttyUSB*, event* ...
ENV{key}	Eventdatenbank	ID_BUS, ID_MODEL, eigene Werte
ATTR{file}	sysfs	aktuelles Attribut des Devices

Matchschlüssel müssen stabil genug sein, um Geräte über Neustarts und Anschlussreihenfolgen korrekt zuzuordnen.

Vier Regeln verhindern fragile Geräteintegration

01 Uevent ist kein Zustandsspeicher

Nachrichten können vor der Beobachtung bereits vergangen sein.

02 Reihenfolge kann variieren

Regeln dürfen keine feste zeitliche Nähe anderer Events voraussetzen.

03 Berechtigungen sind Sicherheitslogik

MODE, OWNER, GROUP und TAG bewusst begrenzen.

04 Namen und Symlinks stabil wählen

Hardwarepfad und Anschlussreihenfolge nicht unnötig fest codieren.

Robuste Integration liest Zustand aus sysfs, behandelt Events idempotent und hält Policy nachvollziehbar.

Vier Werkzeuge machen Zustand und Ereignisse sichtbar

```
$ udevadm monitor --kernel --property  
$ udevadm info --query=all --name=/dev/$DEV  
$ udevadm test /sys/class/block/$DEV  
$ readlink -f /sys/class/block/$DEV/device  
$ cat /sys/class/block/$DEV/uevent
```

1

sysfs

aktuellen Zustand und Beziehungen lesen

2

monitor

Kernel-Uevents mit Umgebung beobachten

3

info

udev-Datenbank und Eigenschaften anzeigen

4

test

Regelauswertung für einen sysfs-Pfad nachvollziehen

Nächste Einheit

Praxis: Gerätemodell und sysfs auf einem laufenden System untersuchen